

Migration Plan: Integrating Refactored Parquet Code with Systematic Caching

Current State Analysis

- **Existing:** `tdata_py/parquet.py` with monolithic functions
- **R Integration:** R code imports via `reticulate::import("tdata_py.parquet")`
- **Dependencies:** Functions rely on `core.py` and `IB_connection.py`

Updated Integration Strategy: Systematic Caching Focus

Phase 1: Add New Files with Systematic Caching (No Disruption)

Goal: Add refactored modules with automatic caching, no context complexity

Actions:

1. Add new files to `tdata_py/` directory:
 - `systematic_caching.py` (replaces `chains_manager.py`)
 - `parquet_storage.py`
 - `parquet_utils.py`
 - `focused_historical.py` (for future historical data)
2. **Keep existing `parquet.py` unchanged** initially
3. Update `__init__.py` to import from new modules with fallback to old

File Structure After Phase 1:

```
tdata_py/  
├── __init__.py      # Updated with new imports + fallbacks  
├── _core.py         # Existing (unchanged)  
├── IB_connection.py  # Existing (unchanged)  
├── parquet.py       # Existing (unchanged initially)  
├── systematic_caching.py  # NEW - core logic with auto-caching  
├── parquet_storage.py  # NEW - storage operations  
├── parquet_utils.py   # NEW - utility functions  
├── focused_historical.py # NEW - future historical data module  
├── contract.py       # Existing  
├── account.py        # Existing  
└── ...               # Other existing files
```

Key Feature: Every IBKR fetch automatically caches data, eliminating redundant API calls

Phase 2: Create Systematic Caching Wrapper

Goal: Seamless transition with enhanced caching behavior

Actions:

- 1. Convert `parquet.py` to systematic caching wrapper:

```
python
```

```
# parquet.py - Systematic Caching Wrapper
```

```
"""
```

Backward compatibility wrapper with systematic caching.

All IBKR fetches automatically cache data for future use.

```
"""
```

```
# Import systematic caching functions
```

```
from .systematic_caching import (  
    getChains as _getChains,  
    getChain as _getChain,  
    getOptionStrikes as _getOptionStrikes,  
    getAllStrikes as _getAllStrikes,  
    getStrikesInRange as _getStrikesInRange,  
    initialize_chains_for_symbols  
)
```

```
from .parquet_utils import (  
    view_parquet,  
    list_parquet_files,  
    cleanup_all  
)
```

```
# Maintain exact original signatures
```

```
def getChains(sym, secType=None, currency=None, exchangeSec=None,  
              exchangeOpt=None, ib_connection=None):  
    """Original signature - now with systematic caching."""  
    return _getChains(sym, secType, currency, exchangeSec,  
                      exchangeOpt, ib_connection)
```

```
def getChain(sym, secType=None, currency=None, exchangeSec=None,  
             exchangeOpt=None, tradingClass=None):  
    """Original signature - now with systematic caching."""  
    return _getChain(sym, secType, currency, exchangeSec,  
                    exchangeOpt, tradingClass)
```

```
def getOptionStrikes(sym, trading_class, expiration, strike_min=None,  
                    strike_max=None, secType=None, currency=None,  
                    exchangeSec=None):  
    """Original signature - now with systematic caching."""  
    return _getOptionStrikes(sym, trading_class, expiration,  
                             strike_min, strike_max, secType,  
                             currency, exchangeSec)
```

```
# Enhanced functions
```

```
def getAllStrikes(sym, trading_class, expiration):  
    return _getAllStrikes(sym, trading_class, expiration)
```

```
def getStrikesInRange(sym, trading_class, expiration, center_strike, range_pct=0.1):  
    return _getStrikesInRange(sym, trading_class, expiration, center_strike, range_pct)  
  
# NEW: Batch initialization for systematic setup  
def initializeWatchlistChains(symbols):  
    """Initialize chain data for multiple symbols at once."""  
    return initialize_chains_for_symbols(symbols)
```

Phase 3: Comprehensive Testing with R Test Suite

Goal: Validate systematic caching and data integrity

R Test Suite (see separate test code section below)

Phase 4: Historical Data Integration (Future)

Goal: Add historical options data capabilities

Actions:

1. Integrate `focused_historical.py` for time-series data
2. Add R functions for historical data graphing
3. Watchlist management for systematic updates

Risk Mitigation

Rollback Strategy

- Keep original `parquet.py` as `parquet_legacy.py`
- If issues arise, quickly revert `__init__.py` imports
- No R code changes needed for rollback

Gradual Adoption

- Phase 1-2: Zero risk - R code unchanged
- Phase 3: Low risk - same interface, better backend
- Phase 4: Optional - only add new features if desired

Dependency Management

- New modules use same dependencies (`core.py`, `IB_connection.py`)
- No additional Python packages required
- Same error handling patterns

Benefits Timeline

Immediate (Phase 1-2):

- Cleaner, testable code architecture
- Better maintainability for future development
- No disruption to existing functionality

Short-term (Phase 3):

- Improved performance through better caching
- More robust error handling
- Enhanced debugging capabilities

Long-term (Phase 4):

- Efficient batch processing for R parallel code
- Advanced file management capabilities
- Foundation for future enhancements

Implementation Checklist

- ☐ **Phase 1:** Add new module files
- ☐ **Phase 1:** Update `__init__.py` with fallback imports
- ☐ **Phase 1:** Test R code works unchanged
- ☐ **Phase 2:** Convert `parquet.py` to wrapper
- ☐ **Phase 2:** Test all R functions still work
- ☐ **Phase 3:** Performance and data integrity testing
- ☐ **Phase 3:** Error handling validation
- ☐ **Phase 4:** Consider new R interface functions (optional)

This approach gives you all the benefits of clean architecture while maintaining 100% compatibility with your existing R integration.